

Q1. A Database Management System (DBMS) differs from a traditional file system in that it stores and manages data in a structured way while providing features such as data integrity, security, concurrent access, and efficient querying. In contrast, a file system stores data in separate files without built-in mechanisms to manage relationships or prevent redundancy. For example, a university managing thousands of student records, courses, and enrollments would benefit greatly from a DBMS because it can maintain consistency across related data and allow multiple users to access the database simultaneously without conflicts.

Q2. A strong entity is an entity that can be uniquely identified by its own primary key, whereas a weak entity cannot be uniquely identified without being associated with another entity known as its owner. The primary key of a weak entity is formed by combining the primary key of the owner entity with the weak entity's partial key. The relationship that links a weak entity to its owner and enables its identification is called an identifying relationship.

Q3. The phone number attribute should be modeled as a multivalued attribute because a single student may have more than one phone number associated with them. A simple attribute can store only one value, while a composite attribute is used when an attribute can be broken down into smaller meaningful parts, such as an address. Since multiple phone numbers can belong to the same student, a multivalued attribute best represents this situation.

Q4. A super key is any set of one or more attributes that can uniquely identify a record in a relation, while a candidate key is a minimal super key, meaning that no attribute can be removed from it without losing its uniqueness. In the relation Student(roll_no, email, name, branch), assuming both roll_no and email are unique, the candidate keys are {roll_no} and {email}. Examples of super keys that are not candidate keys include {roll_no, name} and {email, branch}, since they also uniquely identify records but contain extra attributes that are not necessary for uniqueness.

Q5. If a row is inserted into the Enrollment table with a student_id that does not exist in the Student table, the foreign key or referential integrity constraint is violated. This constraint ensures that every reference made in one table corresponds to an existing record in the related table, thereby maintaining consistency between tables. As a result, the database management system should reject the insertion and generate an error to prevent invalid or orphaned records from being created.

Q6. When mapping a many-to-many (M:N) relationship from an ER diagram to a relational schema, an additional table, often called a junction or associative table, must be created to represent the relationship. The primary key of this table is typically formed by combining the primary keys of the participating entity tables. This relationship cannot simply be represented as a column in one of the entity tables because each entity can be associated with multiple instances of the other entity, and storing multiple values in a single column would violate the principles of the relational model.

Part A – Entity and Relationship Identification

The main entities in the online food delivery platform are Customer, Restaurant, Order, and MenuItem. The Customer entity may have attributes such as CustomerID, Name, PhoneNumber, and Address, while the Restaurant entity may have RestaurantID, RestaurantName, Address, and ContactNumber. The Order entity can include OrderID, OrderDate, TotalAmount, CustomerID, and RestaurantID, and the MenuItem entity can include MenuItemID, ItemName, Price, and RestaurantID. The relationship between Customer and Order is 1:N because a customer can place many orders but each order belongs to only one customer. The relationship between Restaurant and Order is 1:N because a restaurant can receive many orders while each order is associated with exactly one restaurant. The relationship between Restaurant and MenuItem is 1:N because a

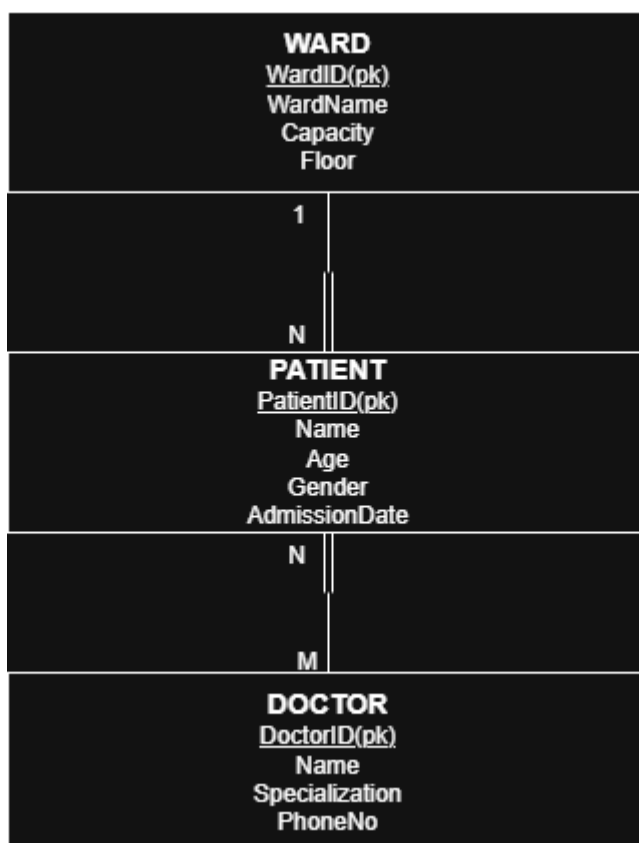
restaurant can offer multiple menu items but each menu item belongs to one restaurant. The relationship between Order and MenuItem is M:N because an order can contain many menu items and the same menu item can appear in many different orders.

Part B – Attribute Classification

A suitable **multivalued attribute** is PhoneNumber because a customer may have more than one phone number associated with their account, allowing multiple values for the same attribute. A suitable **composite attribute** is Address because it can be divided into smaller meaningful components such as House Number, Street, City, State, and PIN Code, making it a combination of several related sub-attributes.

Part C – Relational Schema Design

Since the relationship between Order and MenuItem is many-to-many, a junction table is required to represent it. The relational schema is OrderItem(OrderID, MenuItemID, Quantity), where (OrderID, MenuItemID) forms the composite primary key. OrderID is a foreign key referencing Order(OrderID), MenuItemID is a foreign key referencing MenuItem(MenuItemID), and Quantity is an additional attribute that stores the number of units of a menu item included in a particular order.



The Doctor–Patient relationship is many-to-many (M:N) because a single doctor can treat multiple patients, while a single patient may be treated by multiple doctors during their stay in the hospital. Since relational databases cannot directly represent an M:N relationship, an associative entity called **Treatment** (or DoctorPatient) is created to link doctors and patients. The composite primary key of

this associative entity consists of DoctorID and PatientID, which together uniquely identify each treatment relationship. Regarding participation constraints, the **Patient–Ward** relationship has total participation on the Patient side because every admitted patient must be assigned to a ward, while the Ward side has partial participation since a ward can exist even if it currently contains no patients. Similarly, in the **Doctor–Patient** relationship, the Patient entity has total participation because every patient should be under the care of at least one doctor, whereas the Doctor entity has partial participation since a doctor may temporarily have no assigned patients. One attribute design decision I made was to model **Address** as a composite attribute because it can be divided into meaningful subcomponents such as street, city, state, and postal code. Storing these components separately improves data organization and makes searching, sorting, and reporting more efficient.